

Vikings OOP Mini Project

A short presentation about the historical background, the class design, and what I learned from the project.

Vikings OOP Mini Project



Agenda

- Historical background
- Class relationships
- OOP concepts learned
- Challenges and results

Historical Background: Vikings vs. Saxons

The project is fictional, but it is inspired by a real historical conflict in early medieval England.

Late 700s

- Viking raids began along the coasts of Britain and Ireland.

865

- A large Viking force, often called the Great Heathen Army, landed in East Anglia.

867

- Vikings captured York, an important city in Northumbria.

878

- Alfred of Wessex defeated Guthrum at Edington, leading toward peace terms.

Danelaw

- Northern and eastern England came under strong Scandinavian legal and political influence.

Why it matters here

The code turns a large historical conflict into small objects and methods.

From History to Python Objects

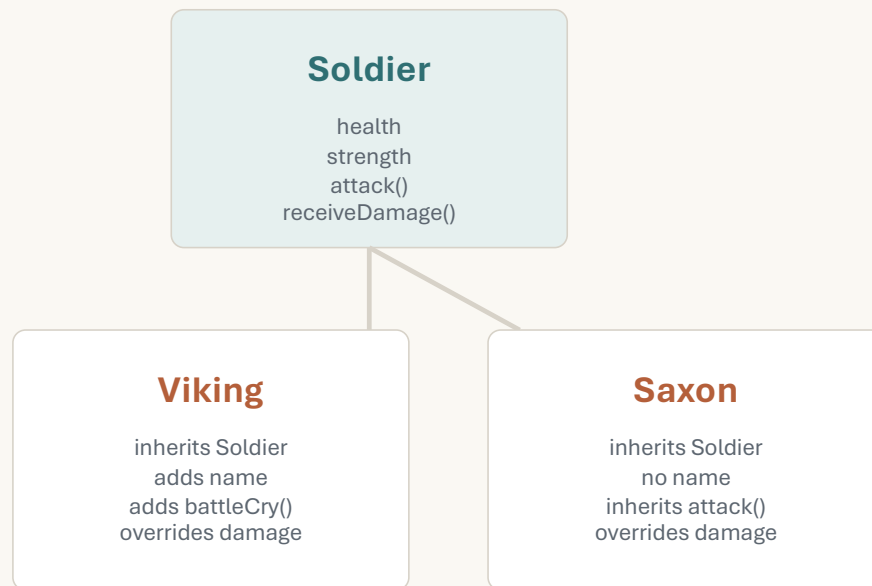
The assignment simplifies the historical setting into a small battle simulation. Each real-world idea becomes a class, an attribute, or a method.

Historical idea	Python design
Individual warrior	Soldier object with attributes health and strength
Viking fighter	Viking subclass with name and battleCry()
Saxon fighter	Saxon subclass with its own damage message
Battle between two groups	War object with two army lists and attack methods

Class Relationships

There are two different relationships in the project: inheritance and composition.

Inheritance



Composition



What We Learned

```
class Viking(Soldier):  
    def __init__(self, name, health, strength):  
        self.name = name  
        super().__init__(health, strength)  
  
    def receiveDamage(self, damage):  
        self.health -= damage  
        return message
```

class

Defines a new type of object and groups related data and behavior.

__init__

Runs when an object is created and sets initial attributes.

super()

Calls the parent constructor so inherited setup is reused.

inheritance

Lets Viking and Saxon reuse Soldier.attack().

overriding

Lets each subclass customize receiveDamage().

What We Learned

```
#Viking
class Viking(Soldier):
...
    def battleCry(self):
        # your code here
        return("Odin Owns You All!")
```

class

Viking inherits from Soldier

battleCry()

Returns the battle cry.

super()

attack()
receiveDamage()
health
strength

inheritance

Lets Viking and Saxon reuse
super() methods.

overriding

Lets each subclass customize
receiveDamage().



```
viking.battleCry()  
-> "ODIN OWNS YOU ALL!"
```

THE PSYCHOLOGICAL SHOCK ASSAULT

Viking warbands were not standard armies; they were apex predators acting on lethal religious fervor. The scream was the first weapon deployed to break enemy morale before the iron ever swung.

Challenges and Final Result

Main challenges

- Choosing what belongs in the parent class versus the subclasses
- Making return strings match the unit tests exactly
- Removing defeated soldiers from the correct army list

Final result

- Soldier tests passed
- Viking tests passed
- Saxon tests passed
- War tests passed



Big takeaway: OOP helps separate shared logic from special behavior, which makes the code easier to test and extend.

The background is a dark, textured surface. In the center, a large, bright red blood splatter is visible. Overlaid on this are two crossed weapons: an axe on the left and a spear on the right. The axe has a weathered metal head and a wooden handle wrapped in leather. The spear has a long, pointed metal shaft and a wooden handle also wrapped in leather. The text is centered over the blood splatter.

Dankeschön
Muchas Gracias
谢谢 (xiexie)
Thank you